

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу «Компьютерная графика»

Основы 3D графики

Вариант: правильный октаэдр

Выполнил: А. А. Устинов

Группа: М8О-307БВ-24

Преподаватель: В. Д. Бахарев

Москва, 2026

1. Условие

Цель работы — познакомиться с основами трёхмерной графики: построением простых 3D-объектов, проекцией трёхмерной сцены на двумерную плоскость экрана, а также научиться работать с матрицами перспективной и ортогографической проекций и матрицами аффинных преобразований.

Вариант задания — построить и вывести на экран правильный октаэдр. Программа должна работать в реальном времени с использованием Vulkan, GLFW и библиотеки интерфейса ImGui и допускать динамическую смену проекции и преобразований объекта.

Выполнены 5 дополнительных заданий:

1. переключение матрицы проекции (ортогографическая и перспективная) в интерфейсе;
2. элементы интерфейса для изменения позиции, поворота и растяжения фигуры;
3. движение и поворот фигуры по сложной траектории, пауза и воспроизведение анимации, изменение скорости и параметров траектории;
4. изменение цвета фигуры с помощью элемента ColorEdit;
5. процедурное назначение цветов вершинам в зависимости от их позиции в локальной системе координат; выбранный в интерфейсе цвет умножается на цвет вершины.

2. Метод решения

2.1. Однородные координаты и аффинные преобразования

Объект описывается один раз в собственной (локальной) системе координат, а затем должен быть размещён в сцене, снят виртуальной камерой и спроецирован на плоскость экрана. Каждое из этих действий — преобразование координат вершины.

Повороты и масштабирование линейны и выражаются матрицей 3×3 . Перенос линейным преобразованием не является: сдвиг на вектор $\mathbf{t}$ нельзя записать как умножение вектора на матрицу 3×3 . Поэтому точка записы-

вается в однородных координатах — четырёхмерным вектором $(x, y, z, 1)^T$, а преобразования — матрицами 4×4 :

$$T(\mathbf{t}) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S(\mathbf{s}) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (1)$$

Четвёртая координата решает две задачи. Во-первых, столбец переноса умножается на единицу и просто прибавляется к результату. Во-вторых, после всех преобразований выполняется перспективное деление: аппаратура вычисляет $(x/w, y/w, z/w)$ — именно деление на w даёт эффект перспективы.

Благодаря однородным координатам все преобразования становятся объектами одного типа и перемножаются между собой, поэтому цепочка сворачивается в одну матрицу. Она вычисляется один раз за кадр, а к каждой вершине применяется однократно:

$$\mathbf{v}' = P \cdot V \cdot M \cdot \mathbf{v}, \quad (2)$$

где M — матрица модели, V — матрица вида, P — матрица проекции. Матрица модели собирается как

$$M = T(\mathbf{t}) \cdot R_z(\gamma) R_x(\alpha) R_y(\beta) \cdot S(\mathbf{s}). \quad (3)$$

Порядок существенен: матрицы перемножаются справа налево, поэтому к вершине сначала применяется масштабирование, затем поворот и только потом перенос. При обратном порядке перенос был бы выполнен до поворота и фигура вращалась бы вокруг начала координат сцены, а не вокруг собственного центра.

Камеры как отдельной сущности в математике преобразований нет: вместо перемещения наблюдателя выполняется обратное перемещение всей сцены. Матрица вида V строится по положению камеры $\mathbf{e}$, точке наблюдения $\mathbf{s}$ и вектору «вверх» $\mathbf{u}_0$ через ортонормированный базис камеры:

$$\mathbf{f} = \frac{\mathbf{c} - \mathbf{e}}{\|\mathbf{c} - \mathbf{e}\|}, \quad \mathbf{r} = \frac{\mathbf{f} \times \mathbf{u}_0}{\|\mathbf{f} \times \mathbf{u}_0\|}, \quad \mathbf{u} = \mathbf{r} \times \mathbf{f}. \quad (4)$$

2.2. Матрицы проекций

Перспективная проекция моделирует зрение человека: чем дальше объект, тем меньше он выглядит. Матрица строится по вертикальному углу обзора φ , отношению сторон окна a и расстояниям до ближней и дальней плоскостей отсечения n и f . Обозначив $k = \text{ctg}(\varphi/2)$:

$$P_{\text{persp}} = \begin{pmatrix} \frac{k}{a} & 0 & 0 & 0 \\ 0 & -k & 0 & 0 \\ 0 & 0 & \frac{f}{n-f} & \frac{nf}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}. \quad (5)$$

Ключевой элемент — -1 в последней строке. Он записывает в w величину $-z$, то есть расстояние от камеры до точки. При последующем делении на w экранные координаты уменьшаются обратно пропорционально расстоянию, что и даёт перспективное сокращение.

Ортографическая проекция переносит точки на плоскость параллельно, без схождения линий: размер объекта не зависит от расстояния до камеры. Если h — высота видимой области в мировых единицах, $t = h/2$ и $r = ta$, то

$$P_{\text{ortho}} = \begin{pmatrix} \frac{1}{r} & 0 & 0 & 0 \\ 0 & -\frac{1}{t} & 0 & 0 \\ 0 & 0 & \frac{1}{n-f} & \frac{n}{n-f} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (6)$$

Последняя строка здесь единичная: w остаётся равным единице, деления не происходит, и зависимость размера от расстояния исчезает. Таким образом, различие двух проекций сводится к одному элементу матрицы, и задание 1 реализовано выбором функции, строящей P .

Обе формулы отличаются от приводимых в литературе по OpenGL по двум причинам. Во-первых, в Vulkan ось Y в координатах кадра направлена вниз, тогда как в мировой системе координат она направлена вверх; поэтому обе матрицы содержат знак минус во второй строке, иначе изображение оказалось бы перевёрнутым. Во-вторых, канонический объём видимости в Vulkan ограничен по глубине диапазоном $z \in [0, 1]$, а не $[-1, 1]$, как в OpenGL; отсюда иной вид третьей строки.

Все матрицы реализованы самостоятельно в файле `math.hpp`, без использования сторонних математических библиотек. Хранение — по столбцам, в том же порядке, в котором язык GLSL интерпретирует тип `mat4`.

2.3. Геометрия октаэдра

Правильный октаэдр — правильный многогранник с шестью вершинами, двенадцатью рёбрами и восемью гранями, каждая из которых является равносторонним треугольником. Наиболее удобное представление — вершины на осях координат: $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$, $(0, 0, \pm 1)$. При таком выборе все рёбра равны $\sqrt{2}$, а центр фигуры совпадает с началом координат, что удобно для поворотов.

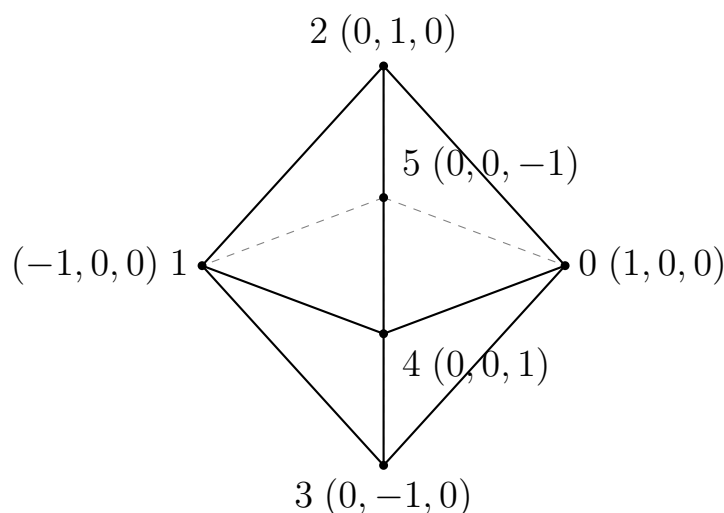


Рисунок 1 — Вершины октаэдра и их номера в вершинном буфере. Пунктиром показаны рёбра, сходящиеся в дальней вершине

Грани задаются индексным буфером: восемь треугольников, по четыре при верхней и нижней вершинах (таблица 1). Использование индексного

буфера позволяет хранить шесть вершин вместо двадцати четырёх: каждая вершина принадлежит четырём граням и описывается один раз.

Таблица 1 – Грани октаэдра и направления их внешних нормалей

Индексы	Нормаль	Индексы	Нормаль
2, 4, 0	(+1, +1, +1)	3, 0, 4	(+1, -1, +1)
2, 1, 4	(-1, +1, +1)	3, 4, 1	(-1, -1, +1)
2, 5, 1	(-1, +1, -1)	3, 1, 5	(-1, -1, -1)
2, 0, 5	(+1, +1, -1)	3, 5, 0	(+1, -1, -1)

Для правильного изображения выпуклого тела используются два независимых механизма. *Буфер глубины*: вместе с цветом для каждого пикселя хранится глубина ближайшего записанного фрагмента, а фрагмент, оказавшийся дальше, отбрасывается. *Отсечение нелицевых граней*: грань, обращённая от наблюдателя, не растеризуется вообще; лицевая сторона определяется по знаку площади треугольника в координатах кадра, то есть по направлению обхода его вершин.

Порядок вершин в каждой тройке выбран так, чтобы обход был против часовой стрелки при взгляде снаружи. Это проверяется по знаку скалярного произведения нормали грани на радиус-вектор её центра: для всех восьми граней произведение положительно, то есть нормали направлены наружу. Инверсия оси Y в матрице проекции переводит мировую систему координат, где ось Y направлена вверх, в систему координат кадра, где она направлена вниз, поэтому направление обхода в терминах Vulkan сохраняется, и в конвейере задано `frontFace = VK_FRONT_FACE_COUNTER_CLOCKWISE`.

2.4. Используемые объекты Vulkan

Для достижения цели работы создаются следующие объекты.

- `VkShaderModule` — два модуля, вершинный и фрагментный шейдеры, загружаемые из скомпилированных файлов SPIR-V. После создания конвейера уничтожаются, так как больше не нужны.

- `VkPipelineLayout` — описывает интерфейс между приложением и шейдерами; в данной работе содержит единственный диапазон push-констант.
- `VkPipeline` — графический конвейер. Существенные настройки: формат вершины из двух величин `VK_FORMAT_R32G32B32_SFLOAT` (позиция и цвет); примитив — список треугольников; отсечение нелицевых граней с обходом против часовой стрелки; включённые тест и запись глубины с функцией сравнения «меньше». Область вывода и прямоугольник отсечения объявлены динамическими и задаются при записи команд, поэтому при изменении размера окна конвейер не требуется пересоздавать.
- `VkBuffer` — вершинный и индексный буферы, размещённые через распределитель памяти `VmaAllocator` в памяти, доступной для записи со стороны процессора. Для объёма в несколько сотен байт промежуточный буфер в памяти видеокарты не требуется.
- `VkRenderPass` и `VkFramebuffer` — проход отрисовки с цветовым вложением и вложением глубины; предоставлены стартовым кодом.

Матрица (2) и цвет передаются в шейдер через push-константы — небольшой блок данных, записываемый непосредственно в командный буфер. Его размер составляет 80 байт (матрица 4×4 из чисел одинарной точности и четырёхкомпонентный цвет), что укладывается в гарантируемые спецификацией 128 байт. Такой способ не требует ни буферов равномерных данных, ни пулов дескрипторов.

2.5. Интерфейс, анимация и цвет

Окно интерфейса разделено на четыре раздела, соответствующих дополнительным заданиям: выбор проекции, преобразования фигуры, анимация и цвет. Значения ползунков разделов **Transform** и **Animation** используются для построения матрицы модели (3): таким образом, задания 2 и 3 сводятся к разным способам собрать M .

Траектория движения (задание 3) — вертикальная восьмёрка, намотанная на окружность:

$$x(t) = R \cos t, \quad y(t) = H \sin 2t, \quad z(t) = R \sin t, \quad (7)$$

где R — радиус окружности, H — амплитуда вертикальных колебаний. Удвоенная частота по оси Y даёт два подъёма за один оборот, то есть траектория не является плоской кривой. Параметр t наращивается на величину $\Delta t \cdot k$, где Δt — длительность кадра, а k — задаваемая в интерфейсе скорость; при паузе наращивание прекращается. Независимо от движения по траектории фигура вращается вокруг собственной оси с задаваемой угловой скоростью.

Цвет вершины (задание 5) вычисляется из её позиции в локальной системе координат при создании вершинного буфера: координата из диапазона $[-1, 1]$ переводится в канал цвета из диапазона $[0, 1]$ по формуле $c = 0,5p + 0,5$. Выбранный в интерфейсе цвет (задание 4) умножается на цвет вершины в вершинном шейдере, а интерполяция между вершинами выполняется растеризатором. Флажок `Procedural vertex colors` заменяет цвета вершин белым, и тогда фигура окрашивается только выбранным цветом.

```

1 #version 450
2
3 layout(location = 0) in vec3 in_position;
4 layout(location = 1) in vec3 in_color;
5
6 layout(location = 0) out vec3 frag_color;
7
8 layout(push_constant) uniform PushConstants {
9     mat4 mvp;    // модель * вид * проекция
10    vec4 tint;    // rgb — цвет из интерфейса, w — использовать ли цвета вершин
11 } pc;
12
13 void main() {
14     gl_Position = pc.mvp * vec4(in_position, 1.0);
15
16     // Задание 5: цвет вершины задан процедурно от её позиции в локальной
17     // системе координат, выбранный в интерфейсе цвет умножается на него
18     vec3 base = mix(vec3(1.0), in_color, pc.tint.w);
19     frag_color = base * pc.tint.rgb;
20 }

```

Листинг 1: Вершинный шейдер: преобразование вершины и расчёт цвета

3. Результаты

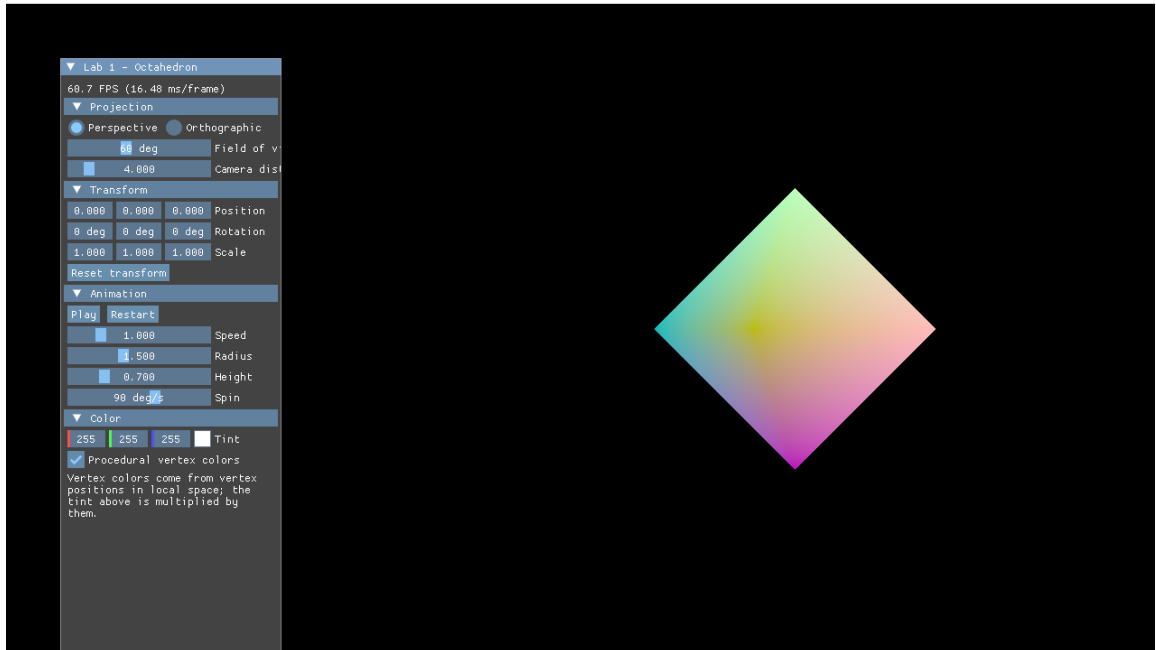


Рисунок 2 – Октаэдр в перспективной проекции. Цвета вершин заданы процедурно от их позиции в локальной системе координат и интерполируются вдоль граней

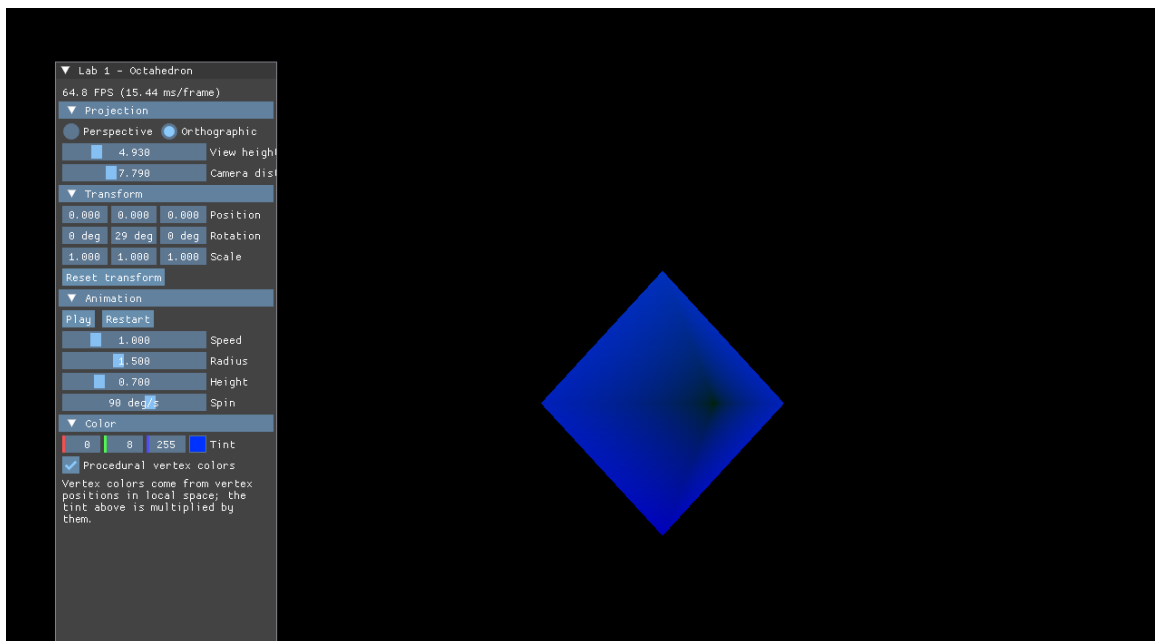


Рисунок 3 – Та же сцена в ортогографической проекции: параллельные рёбра не сходятся, размер фигуры не зависит от удаления камеры

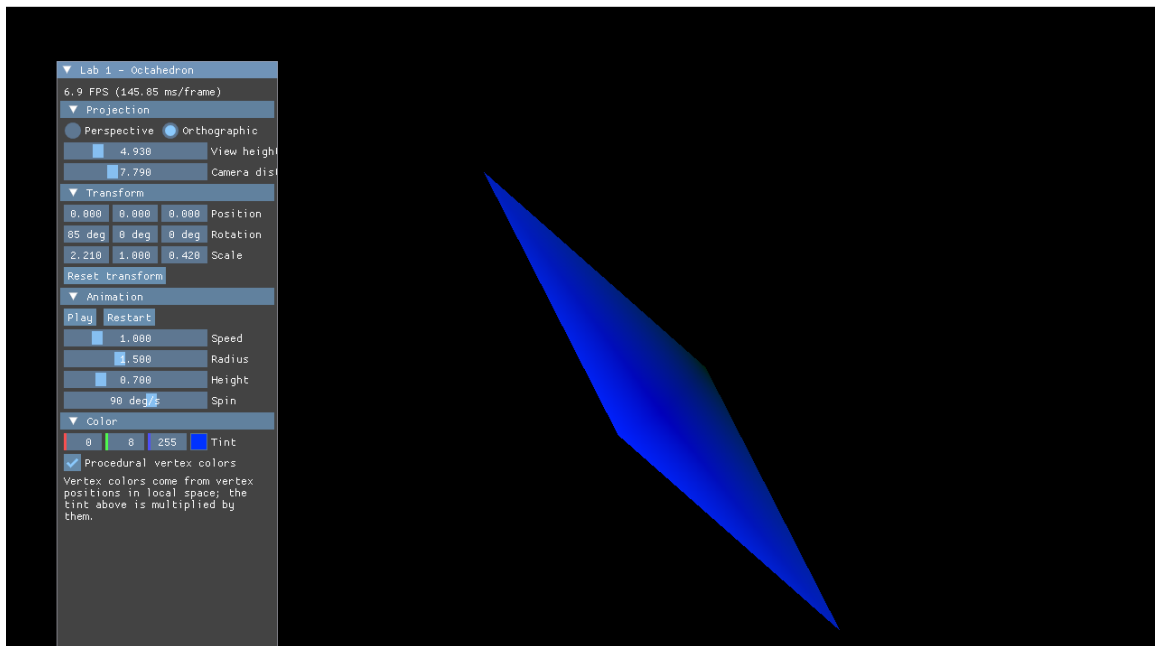


Рисунок 4 – Изменение позиции, поворота и растяжения фигуры ползунками раздела **Transform**

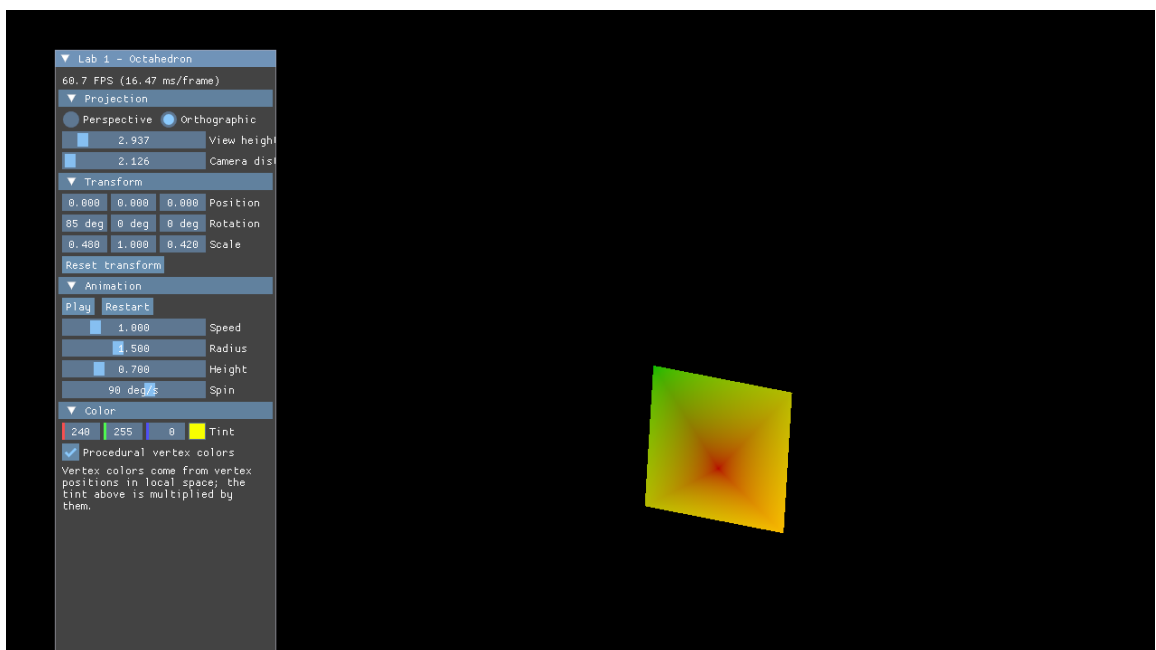


Рисунок 5 – Изменение цвета фигуры: выбранный в ColorEdit цвет умножается на цвета вершин

Различие проекций наиболее заметно, если сместить фигуру от центра экрана и увеличить её: в перспективной проекции грани, расположенные дальше от камеры, сокращаются, а параллельные рёбра сходятся; в ортографической проекции параллельность рёбер сохраняется, и размер фигуры не

зависит от удаления камеры.

4. Выводы

В ходе работы я научился строить простые трёхмерные объекты и выводить их на экран средствами Vulkan в реальном времени, работать с матрицами аффинных преобразований и проекций, а также пользоваться буфером глубины и отсечением нелицевых граней для правильного изображения выпуклого тела.

Главное, что я понял: представление точек в однородных координатах позволяет описать перенос, поворот, масштабирование, видовое преобразование и проекцию объектами одного типа — матрицами 4×4 . Благодаря этому вся последовательность преобразований сворачивается в одну матрицу, вычисляемую один раз за кадр, а каждая вершина обрабатывается единственным умножением. Практическим следствием стало то, что каждое из дополнительных заданий свелось к изменению одной из матриц, а код отрисовки, шейдеры и буферы остались неизменными: переключение проекции — это выбор функции, строящей матрицу P , а преобразования и анимация — разные способы собрать матрицу M .

Отдельно отмечу особенности соглашений Vulkan, которые пришлось учесть: диапазон глубины $[0, 1]$ вместо $[-1, 1]$ и направленная вниз ось Y в координатах кадра. Обе матрицы проекции содержат поправку на это; формулы, взятые из литературы по OpenGL без такой поправки, дали бы перевёрнутое изображение и неверную работу буфера глубины.

Полезным оказался и опыт отладки отсечения граней: ошибка в выборе лицевой стороны не бросается в глаза, поскольку выпуклая фигура остаётся похожей на себя — вместо ближних граней просто отображаются дальние. Обнаружить это удалось только по цветам вершин, так как в центре фигуры оказался цвет дальней вершины вместо ближней.